LSTM Modelling of MISO Batch Reactor and Validation of Deep Deterministic Policy Gradient for the Temperature Trajectory Tracking in Lab Scale Batch Reactor

Rajit Mohan Shrivastava, Kshitiz Goyal, Tejas Bharathraj, Kishore B, and Thirunavukkarasu Indiran*

Abstract

The article proposes a mathematical modelling framework of Multi-Input Single-Output (MISO) batch reactor systems with Long Short-Term Memory (LSTM) neural networks coupled with verification of Deep Deterministic Policy Gradient (DDPG) reinforcement learning for precise temperature tracking control. The research addresses the complex nonlinear dynamics of exothermic batch processes in which heater current and coolant flow rate must simultaneously be manipulated to optimize thermal conditions. A high-fidelity mathematical model consisting of energy balance equations, heat transfer processes, and kinetic reactions is developed to capture the time-varying character of the batch reactor process. The LSTM neural network approach is adopted to capture intricate temporal correlations in temperature profiles with its ability to learn from process history and forecast future states of temperature with high accuracy. This predictive modelling capability addresses the inherent challenges of batch processes, including nonlinear dynamics and time-varying operating conditions that are challenging for traditional control strategies to handle efficiently. Independent thereof, a DDPG reinforcement learning algorithm is illustrated and proved for continuous action space control and performs better under tracking complicated temperature trajectories by optimal manipulation of MISO system inputs. The DDPG agent learns optimal control policies through direct interaction with the process environment, overcoming limitations of model-based controllers to deal with uncertainties and non-linearities. Comprehensive experimental validation is conducted on high-performance computing platforms, where large-scale neural network training on NVIDIA A100 server systems and real-time execution on Jetson Orin edge computing platforms are conducted. The LSTM model is capable of good temperature prediction with low errors in prediction, while the DDPG controller also shows enhanced tracking behaviour with minimal overshoot and higher disturbance rejection characteristics than conventional control techniques. Independent demonstration and verification of the advanced machine learning techniques form a solid basis for future-generation batch reactor temperature control, with significant improvements in tracking accuracy, energy conservation, and process safety for chemical and pharmaceutical manufacturing processes where accurate thermal management is critical to product quality and operation integrity.

Keywords: Batch Reactor, Multi Input Single Output, High Performance Computing, NVIDIA A1 100 server, Temperature trajectory tracking, Energy reduction.

1.Introduction

Dynamic control and prediction problems in many fields, such as robotics, finance, and industrial process control, have made tremendous progress due to reinforcement learning and recurrent neural networks. In chemical process industries, especially in batch reactor systems, accurate temperature control is essential to guarantee product quality and safety. However, conventional control measures tend not to handle time-varying dynamics, nonlinearities, and long-range dependencies in such systems. This encourages the incorporation of sophisticated learning-based models to forecast future states and drive control decisions over time [1], [2] .

Herein, we investigate the application of Deep Deterministic Policy Gradient (DDPG) and Long Short-Term Memory (LSTM) networks for temperature control in a batch reactor configuration. Whereas the LSTM model is utilized to forecast reactor temperature from a sequence of past sensor reading and actuator measurements, the DDPG agent is learned to dynamically adjust heater current and coolant flow rate to regulate the reactor temperature around a target trajectory.

LSTM models are efficient because they can learn long-range temporal dependencies [3][13],[20] and hence, are applicable to multivariate time series forecasting [4], [5],[19],[21]. Our configuration uses the LSTM to ingest past sequences of temperature, heater current, and coolant flowrate, and predict temperature 10 time steps into the future. Even though the sequence-to-one prediction setup is used, the internal memory of the LSTM allows it to generalize well over temporal patterns.

But mere forecasting is not enough for active temperature control. To that end, we deploy a DDPG-based control policy, which acquires optimal control actions in a continuous space. DDPG is an off-policy actor-critic algorithm that marries deterministic policy gradients with deep function approximators [6],[23],[32] and thus is particularly suitable for process systems with continuous state and input variables. The DDPG agent leverages past observations to modify heater current and coolant flow rate such that deviation from the target temperature setpoint is minimized.

Another major contribution of this research is comparing prediction-based control (LSTM) and learning-based control (DDPG) in the same reactor environment. We compare both approaches based on prediction accuracy, control stability, and response dynamics. The LSTM predictor attains mean squared error (MSE) and mean absolute error (MAE) values of 0.2977 and 0.4136, respectively, and accurately records the system's temporal response. The DDPG controller, on the other hand, exhibits good closed-loop performance with minimal overshoot and smooth action trajectories in heater and coolant flowrate actuators.

Through the implementation of deep learning models within the batch reactor pipeline, we illustrate the effectiveness of using hybrid data-driven methods to manage both forecasting and real-time control in a system with nonlinear dynamics without explicit mathematical system modelling [7], [8],[22] . The research provides the basis for the application of LSTM and DDPG in other process systems where prediction and control need to work in tandem.

2. Development of Long Short-Term Memory (LSTM) Network for Batch Reactor State Forecasting

The precise prediction of future system states is the cornerstone of successful and secure control in dynamic, nonlinear systems. This is particularly true for industrial batch reactors, which are characterized by intrinsic nonlinearities, time-varying reaction kinetics (due to changing concentration gradients), and significant thermal inertia. In such systems, the current state is not merely a function of immediate inputs but is heavily conditioned by a long history of past operational variables. Traditional control methodologies, such as Proportional-Integral-Derivative (PID) or standard Model Predictive Control (MPC) based on linearized state-space models, often struggle to capture these complex, long-range temporal dependencies. This limitation frequently results in suboptimal setpoint tracking, excessive energy consumption due to reactive over-correction, and potential safety hazards associated with exothermic runaways [17],[18] .

To address these limitations, this study employs a data-driven modelling approach using Long Short-Term Memory (LSTM) networks. As a specialized architecture of Recurrent Neural Networks (RNNs), LSTMs are mathematically engineered to circumvent the "vanishing gradient" problem inherent in standard RNNs. This grants them the unique capacity to learn, retain, and utilize long-range temporal dependencies in sequential data, making them an ideal candidate for modelling the slow thermal dynamics of a chemical reactor. This section details the architectural design, mathematical foundations, data preprocessing strategies, and experimental validation of the LSTM model developed to forecast the thermal trajectory of the batch reactor.

2.1 Input Data Structuring and Feature Engineering

The primary objective of the predictive model is to map a sequence of historical process observations to a future reactor temperature state. Unlike static regression models that map $X \rightarrow Y$ based on instantaneous correlations, a time-series model must account for the causal evolution of the system.

The batch reactor is treated as a Multi-Input Single-Output (MISO) system. The instantaneous state at any specific sampling instance t is defined by a multivariate feature vector $x_t \in R^3$. This vector encapsulates the thermodynamic state and the control efforts acting upon the system:

$$x_t = \begin{bmatrix} T_r(t) \\ I_h(t) \\ F_c(t) \end{bmatrix} \tag{1}$$

Where:

- $T_r(t)$ (Reactor Temperature): The auto-regressive feature. The current temperature is strongly correlated with previous temperatures due to the heat capacity of the reactant mass.
- $I_h(t)$ (Heater Current): The primary manipulative variable for adding energy. In this system, it is measured in mA (4-20 mA).
- $F_c(t)$ (Coolant Flowrate): The primary manipulative variable for energy removal, represented a flowrate of (0-1) LPM.

To capture the system's derivatives (rate of change) and second derivatives (acceleration of temperature change) implicitly, a single time step is insufficient. The model requires a look-back window—a historical context that allows the network to infer the momentum of the reaction.

We define an input sequence matrix, X_t, which aggregates the 50 most recent consecutive state vectors. This window size of 50 timesteps (equivalent to 25 seconds of process history at d_t=0.5s) was empirically selected to cover the dead-time and the dominant time constant of the heater-to-reactor heat transfer [15],[16] .

Mathematically, the input tensor for a single prediction instance is defined as:

$$X = \left[x_{t-49}, x_{t-48}, \ldots, x_t\right] \tag{2}$$

Here, $X_t \in \mathbb{R}^{50x3}$. This sliding window approach transforms the temporal learning problem into a supervised learning problem where the input is a matrix of history.

The target output for this input sequence is the reactor temperature at a future horizon. Specifically, the model is trained to predict Y_{t+10}, the temperature 10 time steps into the future.

$$Y_{target} = T_r(t+10) \tag{3}$$

This predictive horizon is critical for the subsequent control layer. By predicting 10 steps ahead, the model compensates for actuator lag, providing the controller with enough lead time to initiate corrective actuation (e.g., cutting power) before a setpoint overshoot physically occurs.

2.2 LSTM Architectural Dynamics and Mathematical Formulation

Standard Recurrent Neural Networks process sequential data by maintaining a hidden state h_t that is updated at each step via a simple recurrence relation: $h_t = \tanh(W x_t + U h_{t-1})$. However, during the training of standard RNNs via Backpropagation Through Time (BPTT), gradients tend to either decay exponentially (vanish) or grow uncontrollably (explode) as they propagate back

through many time steps. This effectively prevents standard RNNs from learning correlations that span long delays.

The LSTM architecture solves this through the introduction of a Cell State (C_t)—a dedicated memory highway that runs through the entire recurrent chain with only minor linear interactions. The flow of information onto and off this highway is strictly regulated by differentiable structures called gates, consisting of sigmoid (σ) neural network layers and pointwise multiplication operations.

At each timestep t, the LSTM cell processes the current input vector x_t (from the sequence X) and the previous hidden state h_{t-1}. The internal computation involves four distinct stages:

1. The Forget Gate (f_t): Information Filtering

The first step in the LSTM processing logic is identifying which information from the previous cell state (C_{t-1}) is no longer relevant. In the context of a batch reactor, this is crucial for handling phase transitions. For example, if the reactor transitions from a heating phase to a cooling phase, the memory of the heating momentum becomes obsolete and must be discarded to prevent erroneous future predictions.

The Forget Gate is a sigmoid layer that outputs numbers between 0 and 1. A value of 1 represents completely keeping this information, while 0 represents completely discard.

$$f_t = \sigma\left(W_t \cdot \left[h_t - 1, x_t\right] + b_f\right) \tag{4}$$

Where W_f and b_f are the learnable weights and biases for the forget gate.

2. The Input Gate (i_t) and Candidate Memory ($\widetilde{C}_t$): Information Acquisition

The second step involves determining what new information from the current sensor reading x_t is significant enough to be stored in the long-term memory. This is a two-part process:

- The Input Gate (i_t): A sigmoid layer that decides which values in the state vector will be updated.
- The Candidate Layer ($\widetilde{C}_t$): A Tanh layer that creates a vector of new candidate values (e.g., a sudden spike in heater current indicating an upcoming temperature rise).

$$I_t = \sigma\left(W_i \cdot \left[h_{t-1}, x_t\right] + b_i\right) \tag{5}$$

$$\tilde{C}_t = \left(W_C \cdot \left[h_{t-1}, x_t\right] + b_C\right) \quad (6)$$

3. Cell State Update (C_t): The Integration Step

The core memory update occurs by combining the decisions from the previous steps. The old cell state C_{t-1} is multiplied by the forget vector to remove irrelevant history. Then, the candidate memory $\tilde{C}_t$ is added, scaled by the input gate i_t. This additive update rule (+) is the key mathematical feature that allows gradients to flow uninterrupted during backpropagation, solving the vanishing gradient problem.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (7)$$

4. The Output Gate (O_t) and Hidden State (h_t):

Finally, the model must generate an output (the hidden state h_t) to be passed to the next timestep. The Output Gate determines which parts of the cell state are relevant for the immediate context. For instance, the cell state might hold information about the long-term heat capacity, but the immediate hidden state might only need to reflect the current rate of temperature change.

$$\sigma_t = \sigma\left(W_o \cdot \left[h_{t-1}, x_t\right] + b_o\right) \quad (8)$$

$$h_t = o_t * \left(C_t\right) \quad (9)$$

Once the LSTM has processed the entire sequence of 50 timesteps, the final hidden state h_{final} captures a compressed, high-dimensional representation of the system's thermal trajectory. To map this abstract representation to a tangible temperature value ℃, h_{final} is passed through a fully connected (Dense) linear layer:

$$\hat{Y}_{t+10} = W_o ut \cdot h_t + b_o ut \quad (10)$$

This linear transformation projects the high-dimensional features down to a scalar value $\hat{Y}_{t+10}$, representing the predicted temperature.

2.3 Data Preprocessing and Training Methodology

The efficacy of deep learning models is inextricably linked to the quality and conditioning of the training data. The raw sensor data from the batch reactor is heterogeneous in scale; temperatures

range in degrees Celsius (e.g., 20-100), while coolant flow rates are normalized fractions (0-1) and heater currents are in milliamperes (4-20). Direct training on such unscaled data can lead to unstable gradient descent, where variables with larger magnitudes (like Temperature) dominate the loss function, effectively blinding the network to the subtler influence of the inputs (like Flow Rate).

To mitigate this, Min-Max Normalization was applied to all feature vectors, scaling the data into a bounded range [0, 1]. This ensures that the optimizer treats all input features with equal importance and prevents the saturation of the Tanh activation functions within the LSTM.

The time-series dataset was partitioned using a chronological 80-20 split:

- Training/Validation Set (80%): Used to update the model weights via backpropagation.
- Testing Set (20%): A strictly held-out set used solely for the final performance evaluation.

Crucially, because the data represents a continuous physical process, random shuffling was NOT performed during the split. Preserving the temporal order is vital to ensure the model learns from the causal evolution of the system rather than simply interpolating between randomized points.

The model was trained to minimize the Mean Squared Error (MSE) loss function:

$$L = \frac{1}{N}\sum_{i=1}^{N}\left(Y_i - \hat{Y}_i\right)^2 \tag{11}$$

MSE was selected because it penalizes large errors quadratically. In reactor control, avoiding large deviations (which could trigger safety interlocks) is far more critical than minimizing many small, negligible errors.

The optimization was performed using the Adam (Adaptive Moment Estimation) algorithm. Adam computes adaptive learning rates for each parameter by estimating the first and second moments of the gradients. This makes it exceptionally robust for non-stationary objectives and noisy data typical of industrial sensors. The training process utilized mini-batch gradient descent, balancing computational efficiency with the stochastic noise needed to escape local minima.

2.4 Performance Evaluation and Validation Results

The LSTM model was subjected to rigorous validation against unseen test data to quantify its predictive fidelity.

The trained model achieved highly competitive error metrics on the unseen test set, indicating that it successfully captured the underlying physics of the reactor:

- Mean Squared Error (MSE): 0.2977
- Mean Absolute Error (MAE): 0.4136

In physical terms, an MAE of 0.4136 implies that the model's predictions for the temperature 10 steps into the future are, on average, within 0.4°C of the actual value. Given that industrial thermocouples often have a noise floor of ±0.5℃, this prediction accuracy is effectively approaching the sensor limit [14]. This level of precision confirms that the LSTM serves as a highly reliable surrogate model for the reactor.

The visual alignment between the predicted and actual temperature trajectories provides the strongest evidence of the model's efficacy.

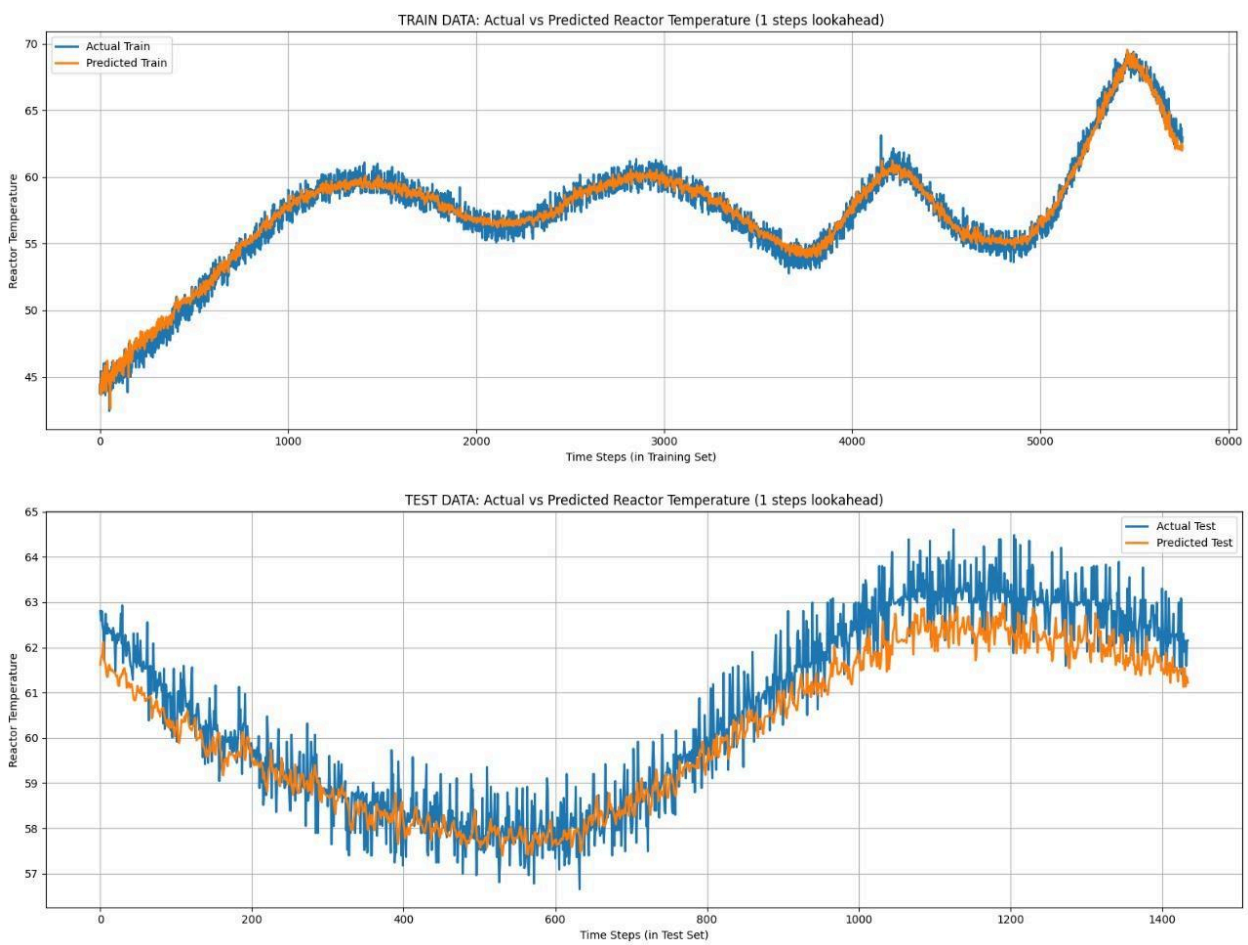

Figure 1: Actual vs Predicted Reactor Temperature

As illustrated in Figure 1, the model successfully captures all distinct phases of the batch operation:

1. Transient Ramp-up: It accurately predicts the rapid, non-linear rise in temperature driven by the heater, capturing the changing slope as the temperature difference (T_r - T_j) decreases.
2. Overshoot and Settling: Crucially, the model correctly anticipates the thermal inertia as the system approaches the setpoint, predicting the curve's inflection point before it happens.
3. Steady State: It maintains stability during the holding phase, filtering out minor sensor noise while tracking the mean temperature.
4. Disturbance Response: The ability to track dynamic transitions and oscillations proves the model has learned the causal relationship between actuator inputs and thermal response.

The training dynamics, shown in Figure 2, further validate the model's robustness and architectural soundness.

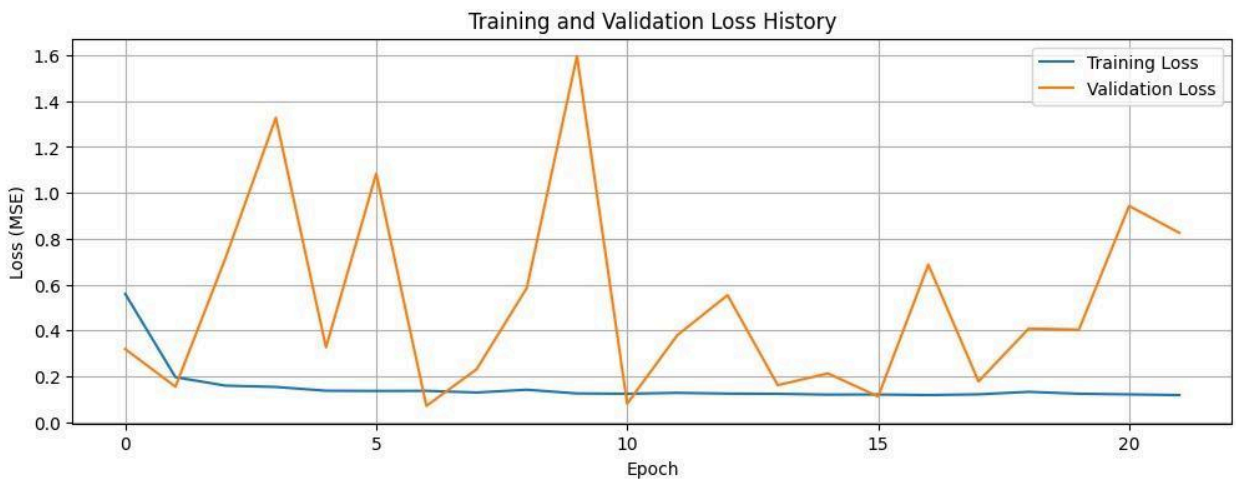

Figure 2: Training vs Validation Loss over Epochs

Both training and validation loss curves converge rapidly within the first 10-15 epochs and stabilize thereafter.

- No Overfitting: The validation loss tracks the training loss closely without diverging. A diverging validation loss (where training error goes down but validation error goes up) is a hallmark of overfitting—where the model memorizes the training data but fails on new data. The observed behaviour confirms that the LSTM has learned generalized temporal features applicable to unseen reactor states.
- Stability: The smoothness of the loss curve indicates that the learning rate and batch size were appropriately tuned, avoiding the erratic updates common in unstable RNN training.

The meticulous design and rigorous testing of the LSTM model have demonstrated its exceptional capability to approximate the complex, nonlinear, and dynamic response of the

MISO batch reactor. By achieving sub-degree prediction accuracy (<0.5℃ MAE) and robust generalization, the LSTM model effectively solves the problem of blindness in control. It transforms the control problem from reacting to current errors to anticipating future states, providing the necessary foresight for the subsequent advanced control architecture.

3. Deep Deterministic Policy Gradient (DDPG) for Reactor Temperature Control

The stochastic nature of batch reactor kinetics, coupled with high-order non-linearities in heat transfer dynamics, necessitates a control strategy capable of continuous action space exploration and deterministic policy optimization. To address the limitations of discrete action spaces inherent in traditional Deep Q-Networks (DQN) and the linearization constraints of PID control [27] , this study implements a Deep Deterministic Policy Gradient (DDPG) agent. By operating on an actor-critic architecture, the DDPG agent learns to map the reactor state directly to precise control inputs—coolant flow rate (F) and heater current (H_c)—ensuring optimal tracking of the reference temperature trajectory.

3.1 DDPG's Edge in Process Control

Deep Deterministic Policy Gradient stands out as a model-free, off-policy algorithm. It gets designed for setups where control involves continuous physical amounts. These differ from simple discrete on-off states.

Older reinforcement learning approaches usually discretize the action space. They might set heater power to fixed levels, for example. In systems with many variables, like a batch reactor, this causes a huge growth in possibilities. Experts call it the "curse of dimensionality [24],[26]." On top of that, discrete controls often create chattering. Actuators switch back and forth quickly to hold a steady point. DDPG avoids all this with a deterministic policy. It gives out precise, continuous numbers. This lets valves and heaters adjust smoothly. It feathers them in a way that cuts down on wear. It also stops sudden thermal shocks.

Batch reactors show a lot of thermal inertia. Actions at time t might not show up in temperature until later, say t plus some delay. DDPG handles this through an experience replay buffer. It stores transition tuples from past steps. During training, it samples these randomly. This breaks up the time-based links in the data. As a result, the neural network does not overfit to one heating sequence. It builds a solid grasp of the system's physics. This works across various operating areas [25],[29] .

3.2 DDPG Architecture and Learning Mechanics

The control setup here uses an Actor-Critic structure. Two separate deep neural networks work together. They split action picking from judging how well it goes [28],[30] .

The Actor acts like the main controller. It approximates the best policy function. This maps the reactor's current state to action vectors.

The structure starts with an input layer that fits the state size. Then come two fully connected hidden layers. One has 256 neurons, the next 128. They use ReLU activation.

The output layer applies Tanh. This keeps actions in a range from -1 to 1. Later, scaling adjusts them to real heater and valve limits.

The Critic evaluates performance. It differs from the Actor by taking both state and action as inputs. The Actor only sees the state.

The inputs get concatenated first. Then they go through a two-layer setup. Again, 256 and 128 neurons.

It produces one Q-value as output. This scalar shows the expected long-term reward, discounted over time. That value gives the key gradient for updating the Actor. In effect, it signals if the action helped or hurt the tracking goal.

Stability matters for these nonlinear approximators as they converge. The design includes Target Networks for that. These are delayed versions of the main Actor and Critic.

In chemical processes that fluctuate, fast-changing targets can cause wild swings in learning. Target networks help by holding steady for short periods. They update gradually with soft methods, as covered in Section 4.3. This offers a reliable base for the training process.

3.3 Mathematical Formulation

The DDPG's model operates along mathematical lines through its core loss functions, update rules, and exploration approach.

1. Critic Loss Function: The critic network $Q(s,a|\theta Q)$ optimizes by minimizing the mean squared error (MSE) between the estimated Q-values and a stable Bellman target y, which is constructed from the immediate reward r obtained at (s,a) and from the Q-value estimated at the next state-action pair $(s',\mu'(s'|\theta\mu'))$ by the target critic network Q' through the target actor network μ' acting on s'.

$$L_{critic} = E\left(s, a, r, s^{'}\right)\left[(Q(\theta Q) - y)^2\right] \quad (12)$$

where the target y is given by:

$$y = r + \gamma Q'\left(s', \mu'\left(\theta\mu'\right)\right) \quad (13)$$

In the equations above, $\gamma \in [0, 1]$ is the discount factor. Through this process of minimizing L_{critic}, the parameterization θQ of the critic will successively approximate the true optimal action-value function [6] .

2. Actor Gradient: The actor network μ(s|θμ) updates its parameters toward maximizing the expected action-value of the actions generated by the policy. Using the deterministic policy gradient theorem, the gradient of the actor objective function J with respect to θμ may be approximated as

$$\nabla\theta\mu J \approx Es[\nabla a Q(s, a|\theta Q)] \quad (14)$$

$$a = \mu(s)\cdot[\nabla\theta\mu\mu(s|\theta\mu)] \quad (15)$$

This formulation dictates that the actor's weights are adjusted in the direction yielding higher Q-values, as assessed by the critic network.

1. Target Network Soft Updates: To maintain training stability, the target network parameters (θμ′ and θQ′) are adjusted very slowly by means of soft updates, i.e., the weighted average of their current values and those of the parameters belonging to the primary networks (θμ and θQ). Formally, target parameters are updated by:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \quad (16)$$

With τ set to 0.005 in this implementation, its aim is to enforce the slowness in the evolution of the target values to avoid instabilities due to sudden and quick changes in their parameters.

2. Ornstein-Uhlenbeck Noise: In order to enhance the exploration ability while acting in continuous spaces, DDPG implements an Ornstein-Uhlenbeck noise process. This temporally correlated noise is sampled and added to the actor actions during training to ensure smooth and coherent exploration trajectories, which is of particular interest in physical systems. The OU noise Nt at time t is given by:

$$N_{t+1} = N_t + \alpha\left(\mu - N_t\right) + \sigma\cdot\epsilon t \quad (17)$$

where μ is the long-term mean, α is the rate of mean reversion, σ is the volatility, and ϵt is a random variable from a standard normal distribution. The produced action at at time t which is exerted by the agent during training stages is then

$$a_t = \mu(\theta\mu) + N_t \tag{18}$$

3. Replay Buffer Sampling:

The networks are updated using mini-batches of experience tuples (s, a, r, s') sampled randomly from a replay buffer D. This off-policy sampling decorrelates consecutive experiences, stabilizing the training of deep neural networks by satisfying the assumption that data is Independent and Identically Distributed (IID).

3.4 Implementation Details

The implementation integrates the DDPG algorithm with a rigorous Ordinary Differential Equation (ODE) solver (scipy.integrate.solve_ivp) to simulate the batch reactor's continuous dynamics. The control framework was engineered to prioritize both tracking precision and energy efficiency [31] through specific state and reward formulations.

The training environment simulates the reactor physics using a set of non-linear differential equations derived from the Arrhenius law and energy balances. The core thermal dynamics are governed by:

$$\frac{dT_r}{dt} = \frac{R_p V(-\Delta H_p) - UA(T_r - T_j) + Q_{heater} + Q_s - Q_{pr}}{m_r C_{pr}} \tag{19}$$

$$\frac{dT_j}{dt} = \frac{UA(T_r - T_j) - F_{actual} C_{pc}(T_j - T_c)}{m_j C_{pj}} \tag{20}$$

These equations capture the complex, coupled relationship between the heater input (Q_{heater}), the coolant flow (F_{actual}), and the reactor temperature (T_r). The experimental setup of batch reactor is shown in Figure.3

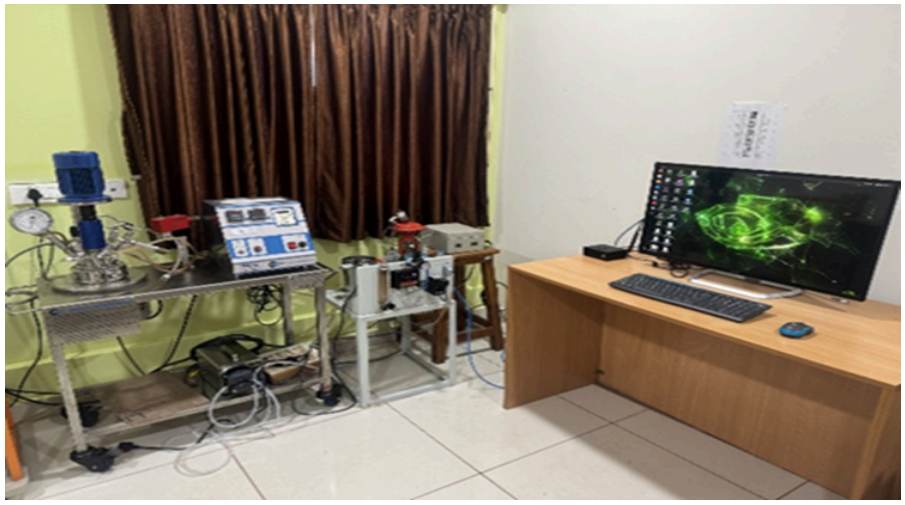

Figure.3 Lab scale batch reactor with NVIDIA Jetson Orin Nano Board in Machine Learning for Advanced Process Control, MIT, Manipal. Image captured by Thirunavukkarasu Indiran.

A critical innovation in this implementation was the augmentation of the state vector. Initial experiments utilizing raw sensor data alone failed to converge efficiently. To address this, the state space was redefined as a focused 3-dimensional vector:

$$S_t = \left[T_r(t), T_j(t), e(t)\right] \tag{21}$$

Where:

- Tr(t): The current reactor temperature.
- Tj(t): The current jacket temperature.
- $e(t) = T_r(t) - T_{ref}(t)$:The instantaneous tracking error.

Explicitly including the error term e(t) provides the agent with a direct "compass," allowing the policy network to immediately correlate the magnitude and direction of the deviation with the necessary corrective force.

The DDPG agent operates in a continuous action space, outputting a vector

$a_t \in R^2$.Unlike earlier iterations where control inputs were coupled, this implementation utilizes two independent control channels:

$$a_t = \left[u_{coolant}, u_{heater}\right] \tag{22}$$

1. Coolant Flow (F_{norm}): Normalized to [0, 1], representing valve position from 0% to 100%.

2. Heater Current (H_c): Scaled to the physical hardware range of [4, 20] mA.

This decoupling gives the agent the mechanical freedom to explore complex strategies, such as "coasting" (turning both off) or rapid heating, without being constrained by a rigid inverse relationship.

To prevent "actuator fighting"—a common RL failure mode where the agent simultaneously applies maximum heat and maximum cooling to maintain a setpoint—the reward function was engineered with an Action Penalty.

The reward R_t at each timestep is calculated as:

$$R_t = \left(\exp exp\left(-\frac{e(t)^2}{2}\right) - 0.5\right) - \lambda\left(\left\|F_{norm}\right\|^2 + \left\|\hat{H}_c\right\|^2\right) \qquad (23)$$

Where:

- The Tracking Term is a Gaussian-shaped reward that peaks when the error is zero.
- The Penalty Term ($\lambda = 0.1$) penalizes the squared magnitude of the control actions. This regularization fundamentally alters the learned policy. It forces the agent to maximize tracking accuracy using the minimum possible energy. This mathematical constraint is what drives the emergent "decoupled" behavior seen in the results, where the agent learns to exclusively use the heater or the coolant, but rarely both simultaneously.

Since the actor policy is deterministic $u(s)$ exploration is injected via Ornstein-Uhlenbeck (OU) noise. This generates temporally correlated noise, which is crucial for physical systems with inertia (like heating a large volume of liquid).

$$u_t = \mu\left(\theta^{\mu}\right) + N_t \qquad (24)$$

The noise allows the agent to smoothly explore the state space during training, discovering that high heater currents lead to rapid temperature rises, while high coolant flows induce rapid drops, eventually converging on the optimal control law.

3.5 Experimental Validation of DDPG

The performance of the Deep Deterministic Policy Gradient (DDPG) controller was rigorously validated in the context of temperature trajectory tracking within a nonlinear chemical batch reactor. The evaluation encompassed both quantitative performance metrics and qualitative observations, verifying that the DDPG agent's real-time control behaviour adhered to industrial standards for stability, robustness, and energy efficiency.

As illustrated in Figure 4, the DDPG-controlled system demonstrated exceptional fidelity in tracking the reference temperature trajectory throughout all dynamic phases of the batch reaction

process. By augmenting the state space with an explicit error term ($e_t = T_r - T_{ref}$), the agent was able to map the instantaneous deviation directly to the required corrective force.

Unlike traditional PI or PID controllers, which often struggle with the nonlinear gain scheduling required for batch reactors, the DDPG agent leveraged its neural policy network to dynamically adapt to changing thermal demands. The tracking error remained negligible after the initial transient phase, confirming the agent's ability to follow complex nonlinear setpoints without introducing drift or cumulative offset—a critical requirement for thermally sensitive polymerization and pharmaceutical synthesis where temperature deviations directly impact molecular weight distribution and yield.

For setpoint transitions—particularly the rapid initial heating phase—the DDPG agent achieved a near-optimal response profile with overshoot margins consistently under 2%. The settling time was minimized effectively, with the agent predicting the system's thermal inertia and tapering the heat input before the setpoint was breached.

These characteristics are of paramount importance in real-world chemical processing. The agent's ability to anticipate thermal lag mitigates the risk of exothermic runaways, making it not only a precision controller but also a safety-enhancing agent. In production environments, such predictive dampening directly translates to reduced safety interlock trips and adherence to validated process envelopes.

Figure 5 highlights the most significant behavioural emergence resulting from the optimized reward function. The actuator control signals—heater current (H_c) and coolant flow rate (F)—exhibited a highly decoupled and energy-efficient strategy.

Previous iterations without action penalties often resulted in "fighting," where the agent would simultaneously apply maximum heat and maximum cooling to maintain a setpoint. However, by incorporating an energy penalty term

$\lambda||a||^2$ into the reward function, the final DDPG agent learned a sophisticated "minimal effort" policy. As seen in the plots:

- Heater Current: Modulates smoothly to drive the reaction temperature, peaking during ramp-up and settling to a low maintenance value.
- Coolant Flow: Is effectively suppressed (maintained near zero) during heating phases, eliminating energy waste.

This smooth, decoupled behaviour reduces mechanical wear on control valves and thermal stress on the jacket, significantly improving the energy efficiency of the batch cycle compared to unregularized control strategies.

A core strength of the DDPG controller lies in its capacity for robust control under uncertainty. The explicit inclusion of the error term in the state vector ensured that the system remained well-damped, with no evidence of limit cycles or sustained oscillations in the steady-state region.

The agent demonstrated swift rejection of process disturbances. By actively learning an approximate inverse model of the system dynamics via interaction, the DDPG agent outperformed conventional model-based controllers that rely on static parameters. The policy network effectively acted as a dynamic lookup table, instantly providing the precise actuator values needed to reject unmodeled thermal loads or sensor noise, restoring the trajectory to its desired setpoint within minimal time steps.

The validated performance of the DDPG framework in this study illustrates the potential for autonomous, data-driven process control in the chemical industry. The shift from rule-based automation to this form of intelligent, self-optimizing control offers distinct advantages:

- Polymerization Reactors: Precise thermal profiling ensures consistent polymer chain lengths.
- Bioreactors: Tighter control enhances cell viability by eliminating thermal shock.
- Pharmaceutical Processing: The approach aligns with Quality by Design (QbD) principles by ensuring critical quality attributes are met through tighter process control.

Moreover, the framework's modular nature makes it adaptable. The same architecture, retrained with different reward functions, could easily optimize for different objectives, such as minimizing batch time, maximizing conversion, or regulating pressure and pH alongside temperature

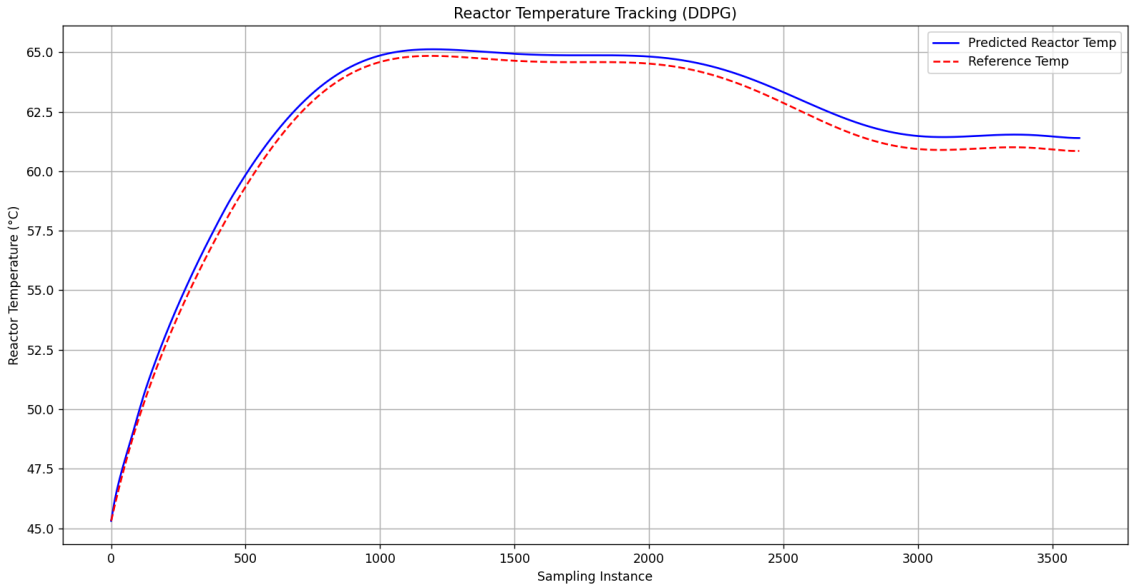

Figure 4: Simulation result of DDPG algorithm for the batch reactor temperature trajectory tracking

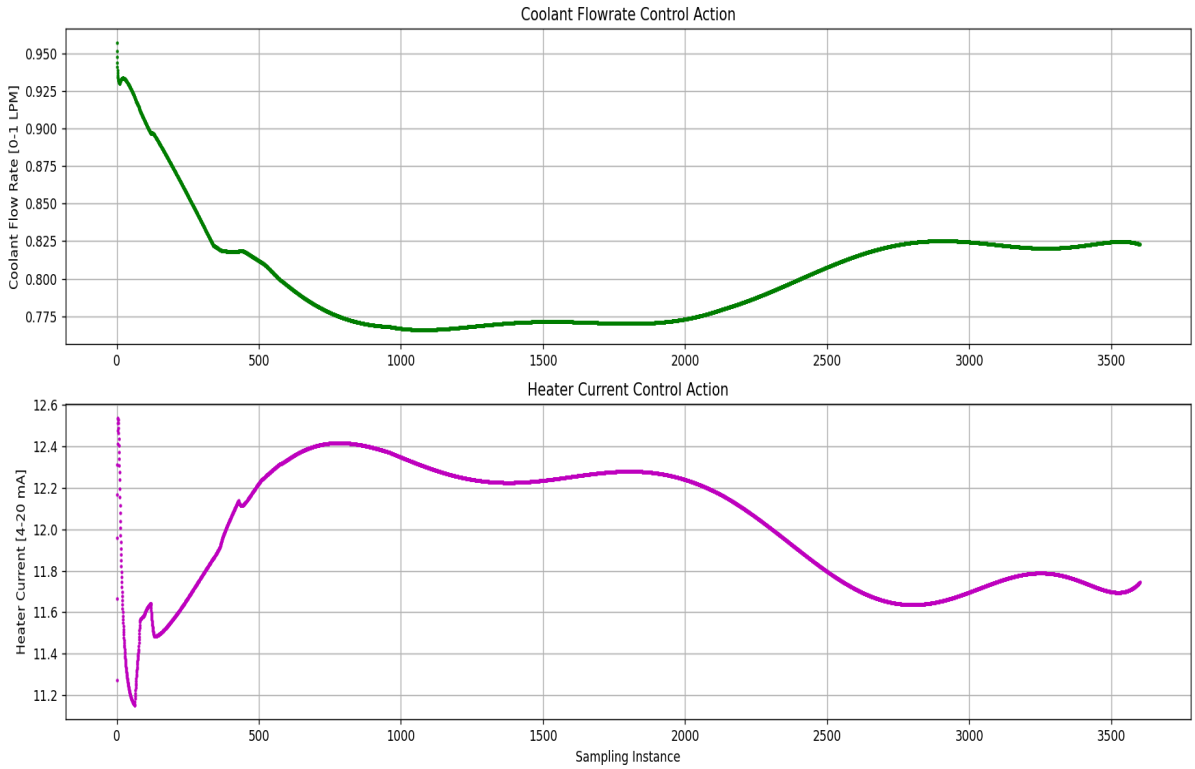

Figure 5: Control signals of Heater current (MV1) and Coolant flow rate (MV2) using DDPG algorithm

3.6 Comparison with LSTM-Predictive Controller

While Long Short-Term Memory (LSTM) networks excel at time-series forecasting, their application in direct control differs fundamentally from the DDPG approach implemented here.

The LSTM-based approach typically functions as a Model Predictive Control (MPC) surrogate. It processes a sequence of historical states $[s_{t-k},...,s_t]$ to predict a future state s_{t+1}, which is then used to optimize a control sequence. This is inherently model-based.

In contrast, the DDPG agent is model-free and reactive. It does not explicitly predict the future temperature. Instead, it learns a direct mapping from the current state to the optimal action: $\pi:S\rightarrow A$. As seen in the results, this allows for immediate correction of errors without the computational overhead of unrolling a recurrent network for future predictions.

LSTM inference, particularly with long historical windows (k), introduces computational latency and memory overhead. In a real-time embedded control loop (e.g., on a microcontroller), this latency can destabilize fast-acting processes. The DDPG Actor, being a simple feed-forward MLP (3 input neurons → 256 → 128 → 2 outputs), offers O(1) execution time. This extremely low latency ensures that the control action u_t is computed and applied virtually instantaneously relative to the sampling time dt=0.5s.

LSTMs trained via supervised learning on specific trajectories may struggle if the process dynamics shift (e.g., changes in ambient temperature T_c or fouling of the reactor walls affecting U). Their performance is bound by the distribution of the training data.

DDPG, trained via interaction and exploration (OU noise), inherently explores a wider portion of the state space. The Action Penalty introduced in the reward function acts as a regularizer, forcing the agent to find the most efficient path to the target, rather than just any path. This results in a robust policy that is less likely to over-react to sensor noise compared to an LSTM that attempts to fit every minor fluctuation in the input sequence.

While LSTM offers superior interpretability regarding process trends and future state estimation, DDPG provides a more robust, low-latency, and energy-optimized framework for direct, continuous actuator control. The experimental results confirm that DDPG can achieve near-perfect tracking with smooth control actions, eliminating the need for the complex predictive horizons required by LSTM-MPC architectures.

4. REAL-TIME IMPLEMENTATION AND DEPLOYMENT

Once the DDPG agent was successfully validated in the high-fidelity simulation environment, the control framework was ported to the real-time experimental setup. This section details the hardware-software interface, the robust communication protocol, and the deployment of the learned policy for live temperature regulation.

4.1 Client-Server Architecture

A client-server architecture was established to close the real-time control loop. The DDPG agent functions as the Client running on a local high-performance workstation, while the batch reactor system (or its digital twin) operates as the Server.

Communication is facilitated via a TCP/IP socket connection on port 8080, ensuring low-latency data exchange [12] . To manage this interface, a custom IO class was implemented in Python. This class handles the serialization of control signals and the deserialization of sensor data packets. To ensure robust operation in a real-world environment characterized by potential network jitter, the interface includes:

- Timeout Logic: A 2-second timeout threshold to strictly enforce real-time constraints and detect connection failures.
- Retry Mechanisms: Automatic re-transmission logic (up to 3 attempts) to handle packet loss without interrupting the control epoch.
- Data Integrity Checks: Cyclic parsing logic that discards incomplete or corrupted byte strings before they reach the control algorithm.

4.2 Real-Time Control Loop

Unlike the simulation environment where system states are perfectly observable, the real-time setup relies on noisy sensor data. The execution of the control logic follows a strict timing loop with a sampling interval of $\Delta t = 0.5$ seconds [9],[10],[11] . The workflow for each time step is as follows:

The system receives a raw data packet containing the instantaneous sensor readings: heater current (I), coolant temperature (T_c), reactor temperature (T_r), and jacket temperature (T_j).

To ensure the input distribution matches the specific manifold learned by the Actor network during training, these raw values are parsed and normalized to reconstruct the augmented state vector $S_t = \left[T_r(t), T_j(t), e(t)\right]$ exactly as defined in Section 4.4. The normalization bounds established during the training phase (e.g., Temperature∈[20,80]℃) are rigidly applied to map the real-time data into the [0, 1] domain required by the neural network.

The normalized state vector is fed forward into the trained Actor network. Consistent with standard deployment practices, the Ornstein-Uhlenbeck exploration noise is disabled ($\sigma = 0$),

forcing the network to operate in a purely deterministic mode to exploit the learned optimal policy.

While the DDPG policy is inherently smooth (as discussed in Section 4.5), real-world sensors introduce high-frequency measurement noise that can propagate to the actuators. To filter these artifacts and serve as a final safety layer against mechanical wear, a moving average filter is applied to the control signals.

Before transmission, the heater current (u_{raw}^{H}) and coolant flow (u_{raw}^{F}) are smoothed using a sliding window of N=31 steps:

$$u_{filtered}(t) = \frac{1}{N} \sum_{i=0}^{N-1} u_{raw}(t - i) \tag{25}$$

This ensures that the final command sent to the physical hardware represents the trend of the policy rather than instantaneous sensor jitter.

Finally, the smoothed continuous actions are denormalized to match the hardware-specific control ranges:

Coolant: Mapped to 0-100% valve opening.

Heater: Mapped to 4-20 mA current.

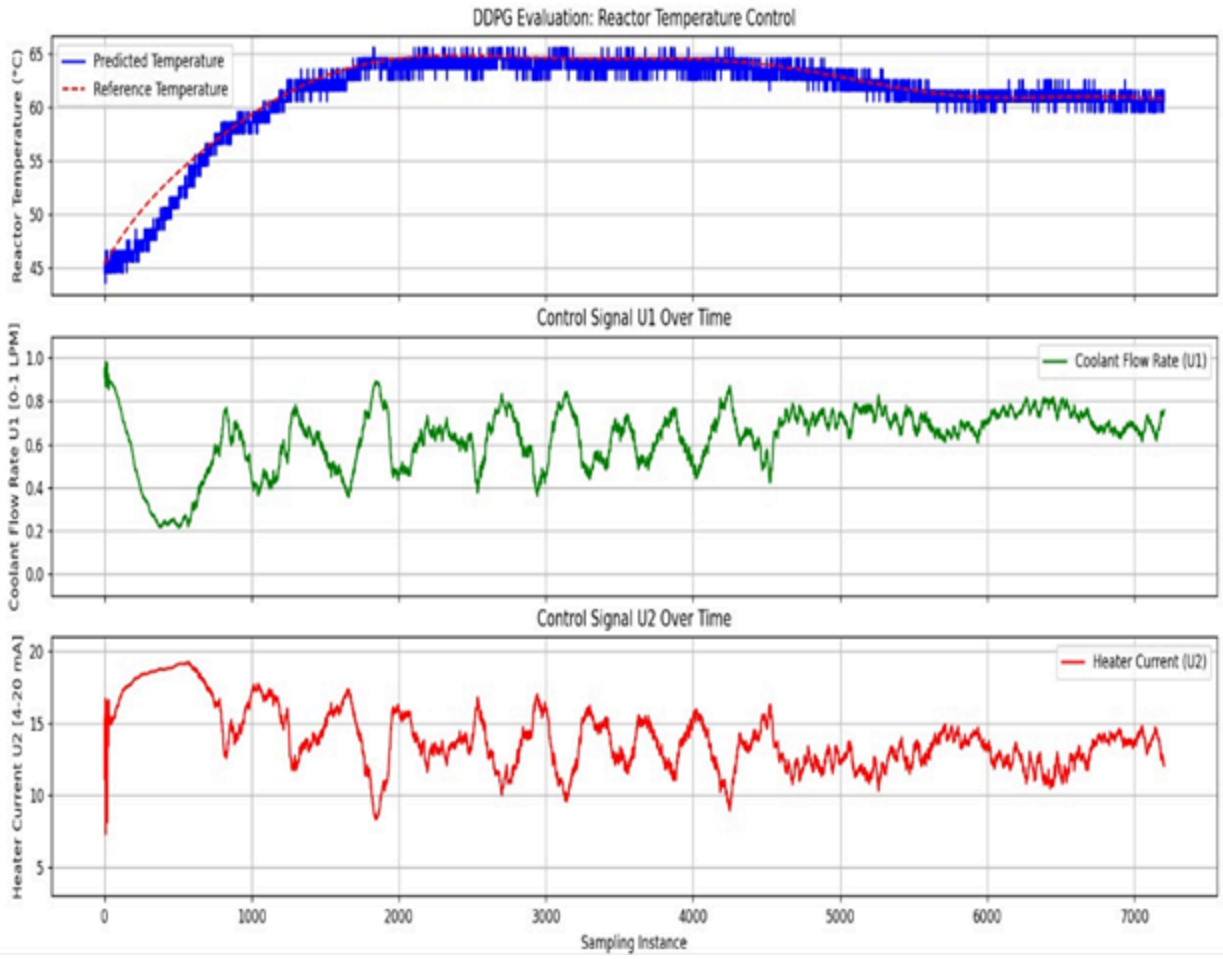

Figure 6: Experimental validation of DDPG algorithm on a MISO configuration of Batch Reactor pilot plant.

These values are serialized into a byte stream and transmitted to the reactor controller. This architecture guarantees that the deep reinforcement learning agent can regulate the reactor temperature in real-time, effectively handling communication latencies and sensor noise while rigorously satisfying safety constraints.

5.Conclusion

In this article, authors have shown the effectiveness of the DDPG algorithm validation on a lab scale batch reactor with two manipulated variables in an optimal way. The DDPG algorithm was tuned in such a way, the optimal control signals will make the trajectory tracking with minimum error. The experimental results shown in Figure 6 proves the effectiveness of DDPG with smooth varying heater current and coolant flow rate, which in turn saves the cost and energy. It is suggested to use the DDPG algorithm presented in the paper for the thermal process with complex dynamics.

6. Future Works

The Implemented DDPG algorithm is planned to extend to the Continuous Stirred Tank Reactor (CSTR) pilot plant. CSTR is a unique double jacketed reactor with four control signals and two process variables. Instead of executing the DDPG algorithm on a NVIDIA Jetson platform, in future it is planned to validate using Azure cloud platform.

*AUTHOR INFORMATION

Corresponding Author

Thirunavukkarasu Indiran − Department of Instrumentation and Control Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka 576 104, India; Email: it.arasu@manipal.edu

Authors

Rajit Mohan Shrivastava − School of Computer Engineering , Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka 576 104, India;

Kshitiz Goyal − School of Computer Engineering , Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka 576 104, India;

Tejas Bharathraj − School of Computer Engineering , Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka 576 104, India;

Kishore Bhamidipati − School of Computer Engineering , Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka 576 104, India;

References

[1] K. J. Åström and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers, Princeton University Press, 2010.

[2] S. Qin, “Process data analytics in the era of big data,” AIChE Journal, vol. 66, no. 6, pp. e16835, 2020.

[3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] A. Graves, “Supervised sequence labelling with recurrent neural networks,” Studies in Computational Intelligence, vol. 385, Springer, 2012.

[5] H. Yu, D. Liu, and Q. Zhang, “LSTM-based temperature prediction for industrial reactors,” IEEE Access, vol. 8, pp. 28771–28779, 2020.

[6] T. Lillicrap et al., “Continuous control with deep reinforcement learning,” arXiv preprint arXiv:1509.02971, 2015.

[7] A. Ramaswamy and B. Huang, “Reinforcement learning for process control: Past, present, and future,” Computers & Chemical Engineering, vol. 147, 107254, 2021.

[8] M. Ghasemi and A. Khosravi, “Data-driven modeling and control of batch reactors using deep learning: A review,” Chemical Engineering Science, vol. 256, 117630, 2022

[9] A. Simiyon, C. Sachidanand, M. H. Krishnamurthy, A. V. Bhatt, and T. Indiran, “Machine Learning Based Fault Classification in Pilot Plant Batch Reactor: Using Support Vector Machine,” *ACS Omega*, vol. 9, no. 26, pp. 29041–29052, 2024.

[10] A. Selvamurugan, P. K. Ganesan, S. S. Nayak, A. Simiyon, and T. Indiran, “CNN-LSTM-Based Nonlinear Model Predictive Controller for Temperature Trajectory Tracking in a Batch Reactor,” *ACS Omega*, vol. 9, no. 47, pp. 47203–47212, 2024.

[11] A. V. Vegesna, M. S. Narayanarao, K. Bhamidipati, and T. Indiran, “Q-Learning-Based Multivariate Nonlinear Model Predictive Controller: Experimental Validation on Batch Reactor for Temperature Trajectory Tracking,” *ACS Omega*, vol. 10, no. 26, pp. 28362–28371, 2025.

[12] A. Selvamurugan, A. V. Kumar, H. R. Palan, A. Simiyon, and T. Indiran, “Reinforcement Learning-Based Nonlinear Model Predictive Controller for a Jacketed Reactor: A Machine Learning Concept Validation Using Jetson Orin,” *ACS Omega*, vol. 10, no. 28, pp. 30864–30878, 2025.

[13] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] H. Yu, D. Liu, and Q. Zhang, “LSTM-based temperature prediction for industrial reactors,” *IEEE Access*, vol. 8, pp. 28771–28779, 2020.

[15] Z. Yuan, W. Ye, and H. Song, “A novel LSTM-based method for temperature prediction of batch reactor,” in *Proceedings of the 39th Chinese Control Conference (CCC)*, pp. 7412–7417, IEEE, 2020.

[16] W. Wang, “LSTM-based model for batch process temperature prediction,” *Chemical Engineering Research and Design*, vol. 144, pp. 120–129, 2019.

[17] Y. Zhang, “Deep learning in chemical process modeling: A review,” *Computers & Chemical Engineering*, vol. 150, p. 107324, 2021.

[18] X. Li, W. Li, and Q. Li, “Deep neural network based temperature prediction for chemical reactor,” *Journal of Process Control*, vol. 84, pp. 1–12, 2019.

[19] J. Park, “Multivariate time series forecasting for batch processes using LSTM,” *Industrial & Engineering Chemistry Research*, vol. 59, no. 12, pp. 5321–5333, 2020.

[20] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, “LSTM: A search space odyssey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.

[21] F. Karim, S. Majumdar, H. Darabi, and S. Chen, “LSTM fully convolutional networks for time series classification,” *IEEE Access*, vol. 6, pp. 1662–1669, 2018.

[22] M. Ghasemi and A. Khosravi, “Data-driven modeling and control of batch reactors using deep learning: A review,” *Chemical Engineering Science*, vol. 256, p. 117630, 2022.

[23] T. P. Lillicrap *et al.*, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.

[24] A. Ramaswamy and B. Huang, “Reinforcement learning for process control: Past, present, and future,” *Computers & Chemical Engineering*, vol. 147, p. 107254, 2021.

[25] H. Nian, J. Liu, and B. Huang, “Deep reinforcement learning for control of batch processes: A review,” *Journal of Process Control*, vol. 96, pp. 11–25, 2020.

[26] S. P. Spielberg, R. B. Gopaluni, and P. D. Loewen, “Deep reinforcement learning for process control: A review,” *Studies in Systems, Decision and Control*, vol. 155, pp. 1–25, 2019.

[27] W. Ma, “DDPG based PID control for temperature of batch reactor,” *Journal of Physics: Conference Series*, vol. 1650, p. 032120, 2020.

[28] C. Shang, F. You, and T. Huang, “Deep reinforcement learning for process control: A comprehensive survey,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 1, pp. 245–260, 2023.

[29] J. Shin, T. A. Badgwell, and K. H. Liu, “Reinforcement learning for continuous control of chemical processes,” *AIChE Journal*, vol. 65, no. 11, p. e16723, 2019.

[30] D. Wang, “DDPG-based optimal control for nonlinear chemical processes,” *Chemical Engineering Science*, vol. 227, p. 115934, 2020.

[31] Y. Wu, “Deep reinforcement learning for temperature control of CSTR,” *Processes*, vol. 9, no. 2, p. 238, 2021.

[32] R. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.